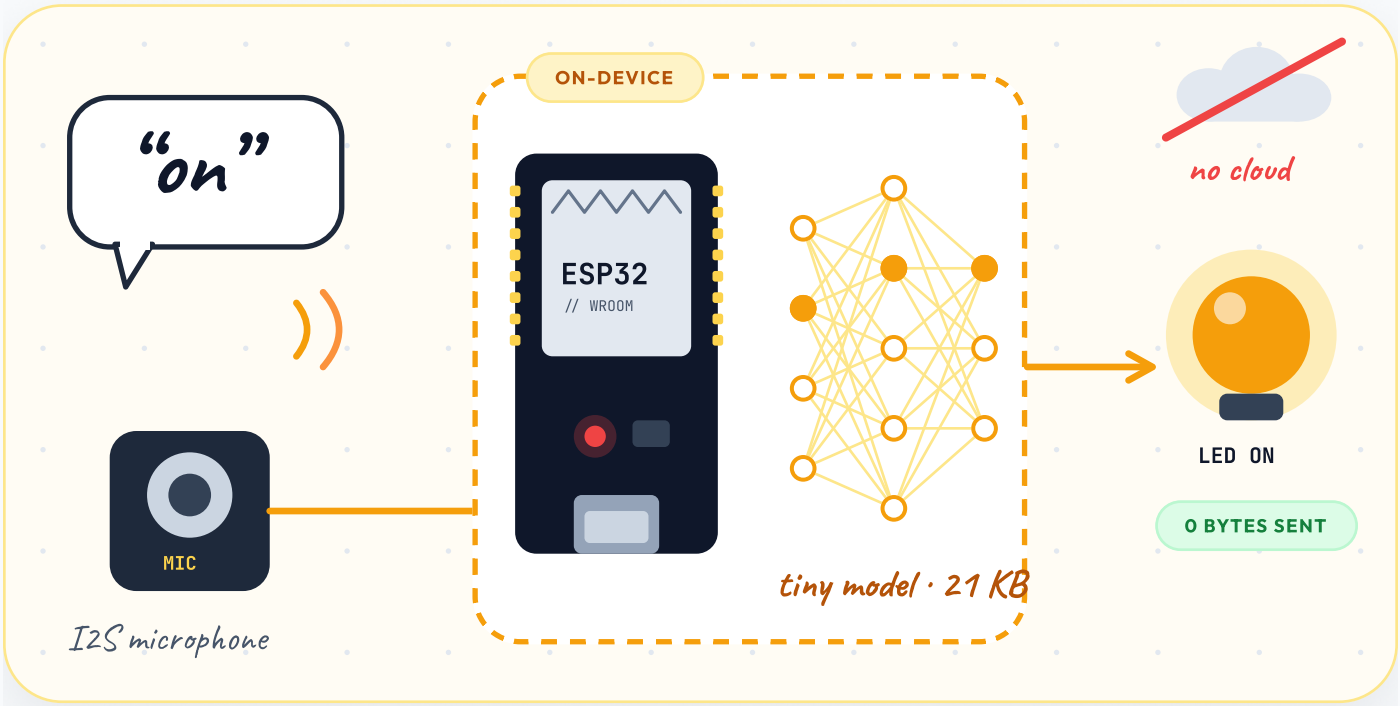
EDGE AI · FREE STARTER KIT

Offline voice AI on a $10 chip.

No internet. No cloud. No per-request fees. Say “on” and a tiny neural network on an ESP32 hears you, understands you and acts, all by itself.



~21 KB
the whole AI model

4× / second
it listens for your words

$0
per request. Forever.

1 Train
Teach a tiny model 8 words on your laptop.

2 Flash
Put the model on the ESP32 with one command.

3 Talk
Say “on” or “off”. The LED obeys.

It understands YES · NO · ON · OFF · UP · DOWN · STOP · GO

Comment EDGE AI on the reel → *repo link lands in your DM*

THE BIG IDEA

How a $10 chip understands you

Normally your voice travels to a cloud server and back. Edge AI keeps the whole trip inside the chip. Here is the difference, and how the chip does it.

The cloud way

SLOW & COSTLY

MIC → WI-FI → SERVER → REPLY

- ✗ Needs internet, every time
- ✗ You pay per request
- ✗ Your voice leaves your device

The edge way

PRIVATE & FREE

MIC → ESP32 → ACTION

- ✓ Works with no internet
- ✓ Zero per-request fees
- ✓ Your voice never leaves the chip

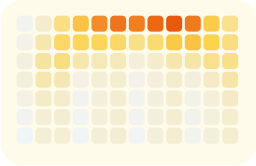
The 4-step pipeline *runs 4 times every second*

1 Listen



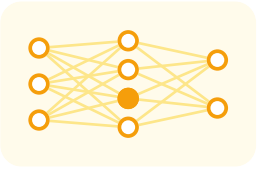
The mic sends 16,000 numbers every second. We keep the last 1 second.

2 Make a picture



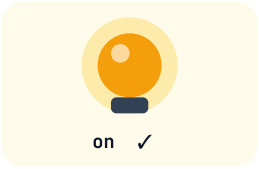
Slice it into 49 pieces and measure 32 pitch bands. Sound becomes a small image.

3 Ask the model

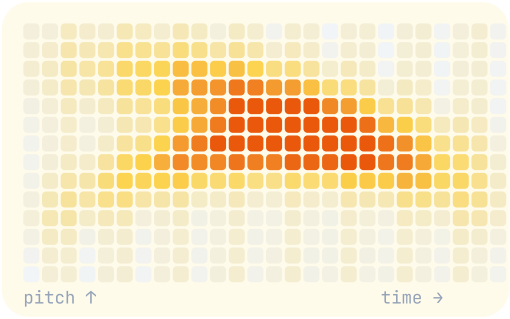


A tiny neural network looks at that image and scores each word.

4 Act, if sure



Needs 80%+ sure, twice in a row. Then 1-second cooldown. Then LED.



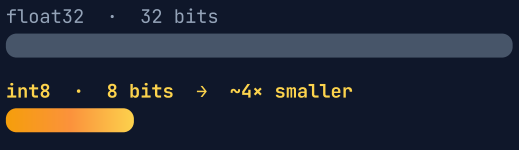
Turn sound into a picture

Think of it as a heat map of your voice. Bright orange means lots of energy at that pitch, at that moment. The word “on” has its own pattern.

Photos and spectrograms are both images, so the same kind of network (a CNN) can read both. It looks for the pattern of each word.

Why int8?

Normal models use 32-bit decimals. We squeeze them to 8-bit whole numbers. The model gets about 4× smaller and much faster, with a tiny accuracy drop. train.py measures that drop for you.



WHAT YOU NEED

Build it in 3 steps

About $10 to $15 of parts and a free afternoon. No soldering needed if you use jumper wires.

$6 – 10
ESP32-S3 dev board
DevKitC-1 or similar

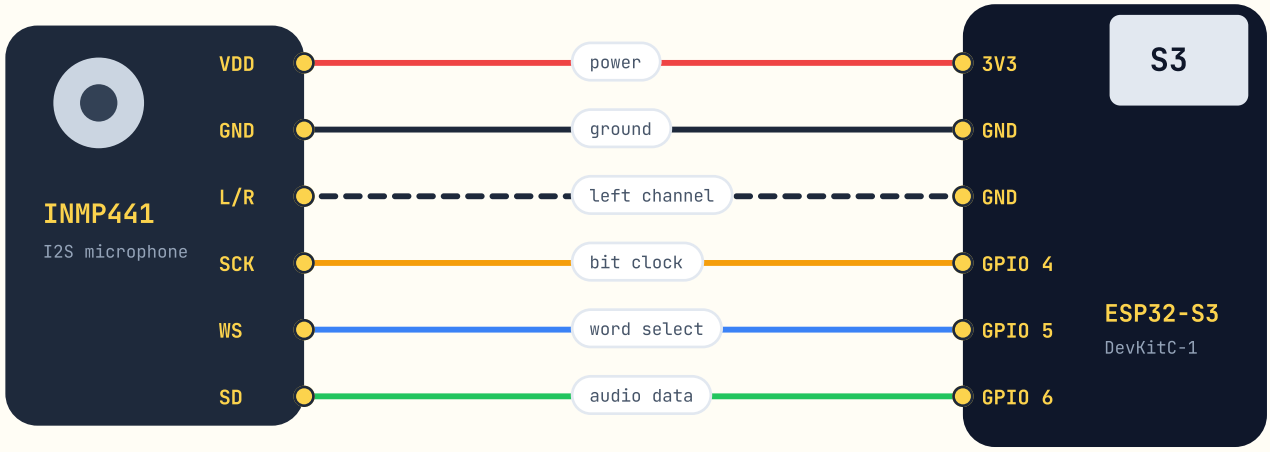
$2 – 5
INMP441 I2S mic
or any I2S mic, same pins

~$1
5 jumper wires
female–female or breadboard

≈ $10 – 15
total in parts
+ laptop, ESP-IDF v6.1 (free)

Wiring

6 wires, left to right. Mic L/R pin goes to GND so it reads the left channel.



1 Train

On your laptop. Downloads Google's Speech Commands (2.3 GB, once), trains a small CNN, shrinks it to int8.

```
terminal · laptop
$ cd train
$ python -m venv .venv && source .venv/bin/activate
$ pip install -r requirements.txt
$ python train.py
# 1-minute check:  python train.py --synthetic
```

2 Flash

Build the firmware and send it to the board. The first build fetches esp-tflite-micro, so be online for that step.

```
terminal · ESP-IDF v6.1
$ cd ../firmware
$ idf.py set-target esp32s3
$ idf.py build flash monitor
```

3 Talk to it

Open the monitor. Say yes, no, on, off, up, down, stop or go, about a hand's distance from the mic.

```
serial monitor
Model ready. Say: on, off, yes, no, up, down, stop, go
I (4210) edgeai: on (93% sure, NN ms per inference)
# example output
```

CUSTOMISE & FIX

Make it yours

Two small edits turn this demo into your project. Change the words it knows, then change what happens when it hears them.

1 Teach it new words

Any word in the Speech Commands set works: left, right, numbers zero to nine and more. Edit the list, retrain, flash.

train/dataset.py

```
COMMANDS = ["yes", "no", "on", "off",
            "up", "down", "stop", "go"]
# try: ["left", "right", "on", "off"]
```

2 Control real things

This is where your project starts. Swap the LED for a relay, a motor driver or a servo.

firmware/main/main.cc

```
static void on_command(const char *word) {
  if (strcmp(word, "on") == 0)
    gpio_set_level((gpio_num_t)PIN_LED, 1);
  if (strcmp(word, "off") == 0)
    gpio_set_level((gpio_num_t)PIN_LED, 0);
}
```

Tuning knobs *all in board_config.h*

SETTING	DEFAULT	WHAT IT DOES
CONFIDENCE	0.80	How sure the model must be. Raise it for fewer false triggers.
CONFIRMATIONS	2	Same word must win this many times in a row.
MIN_RMS	120	Ignores very quiet audio. Saves power, fewer false triggers.
MIC_SHIFT	14	Mic volume. Lower it (12) if it never hears you.
TENSOR_ARENA_BYTES	48 KB	Model memory. Raise it if AllocateTensors fails.
PIN_LED	GPIO 2	Plain LED. Use 48 or 38 for the RGB LED on some S3 boards.

Stuck? Check here first

PROBLEM	FIX
Build says model_data.cc not found	Run step 1 (train) first. It writes the model for you.
AllocateTensors failed	Raise TENSOR_ARENA_BYTES in board_config.h.
Nothing is ever detected	Lower MIC_SHIFT (14 to 12) or MIN_RMS. Check mic L/R is on GND.
Random false triggers	Raise CONFIDENCE to 0.9, or CONFIRMATIONS to 3.
Works on my voice only	Normal for a small model. Try --aug-copies 2, or record your own samples.
Different chip (classic ESP32, C3)	Change idf.py set-target. Pins may differ.

Check the maths

C on chip = Python

The chip computes features in C. Training used Python. If they drift apart, the model quietly gets worse on the board. This test compares them.

```
$ python tests/test_parity.py
```

Tested with

ESP-IDF 6.1 · ESP32-S3 · esp-tflite-micro 1.4.1 · TensorFlow 2.21.0 · Python 3.12

Your Edge AI roadmap

You just ran a neural network on a few-dollar chip. Do these stages in order. Each one builds on the last.

0

Finish this kit — a weekend

Get “on” and “off” working on your own voice · Read features.c and main.cc line by line · Change the word list and retrain once

Goal: You can say why each file exists.

1

Embedded basics, properly — 2–4 weeks

GPIO, timers, interrupts, I2C and SPI with ESP-IDF · FreeRTOS tasks and queues. Why the audio loop must never block · Read a datasheet and wire a sensor from scratch

Goal: You can explain every line in app_main().

2

Add gestures — 2–3 weeks — FROM THE REEL

Add an IMU (like the MPU6050) over I2C · Record wave, circle, shake, tap: 50+ samples each · Train a small 1D CNN, export int8, run it the same way

Goal: One board that reacts to your voice and your hand.

3

Learn the ML side properly — 3–4 weeks

Why spectrograms work. What a convolution really does · Overfitting, validation sets, confusion matrices · Quantisation: measure the accuracy drop every time

Goal: Bad accuracy? You know if it is the data, features or model.

4

Make it production-grade — 4–6 weeks

Memory and latency budgets. Profile before you optimise · Power: deep sleep, wake on sound, battery maths · OTA updates with safe rollback. Test in real, noisy rooms

Goal: It survives outside your desk.

5

Connect, only when it helps — 2–4 weeks

Edge AI does not mean never using the network. The network is optional · Send results (“light turned on”), not raw audio · MQTT to a dashboard, or a cloud backend for fleet management

Goal: Network is a feature, not a crutch.

6

Ship a portfolio project — your pace

Pick one: voice device, gesture remote, predictive-maintenance sensor, keyword-triggered recorder · Write it up with accuracy, memory use and a demo video

Goal: Something real that teams can see.

Want a guided path?

Live sessions, code reviews and real projects in the EdgeAI Engineering Bootcamp.

analogdata.io →